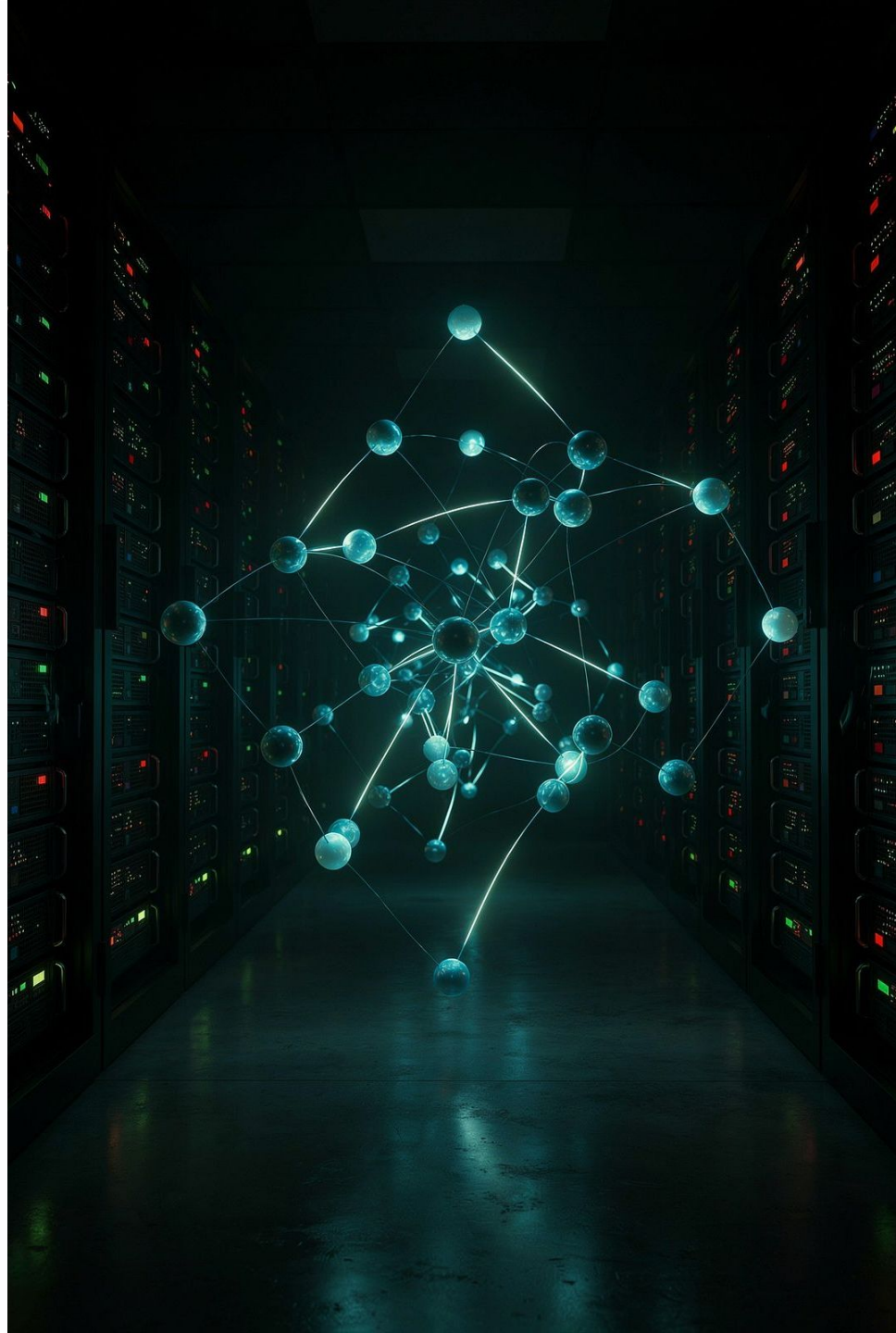
Machine Learning & Analytics in AIOps with GitHub

Module 3 — How AI and machine learning transform the way we operate, monitor, and improve modern software systems. From detecting anomalies before they become outages, to automating workflows with GitHub Actions, this module connects real-world ML concepts to everyday DevOps practice.

MODULE 3 | AIOPS | MACHINE LEARNING



What We'll Cover in This Module

This module takes you on a journey from understanding **why machine learning matters in operations** to actually building, tracking, and automating ML-powered workflows using GitHub. Each topic builds on the last — think of it as learning to drive before hitting the highway.

01

ML Foundations in AIOps

Core concepts: anomaly detection & predictive analytics

02

Practical Algorithms

Classification, regression, and clustering with examples

03

GitHub for ML Deployment

Version control for models, not just code

04

Model Tracking & Automation

Experimentation, evaluation, and GitHub Actions



Why Machine Learning in AIOps?

Modern software systems generate **millions of log events, metrics, and alerts every single day**. No human team can manually read through all of it. That's where AIOps comes in — using machine learning to automatically surface what matters most, so your team can act fast and stay ahead of problems.

Think of AIOps like having a tireless AI assistant watching your systems 24/7, raising its hand only when something truly needs your attention.

Machine Learning Foundations in AIOps

At its heart, AIOps uses machine learning to **learn what "normal" looks like** — and then flag anything that deviates from it. Instead of writing thousands of hand-crafted alert rules, ML models discover patterns in your data automatically and adapt over time as your system evolves.

🔍 Observe

Collect logs, metrics, and traces from systems continuously

🧠 Learn

ML models identify normal behavior and patterns

⚡ Act

Flag anomalies and predict issues before they cause downtime

Anomaly Detection — Catching What Rules Miss

The Problem

Traditional monitoring relies on fixed thresholds — e.g., "alert if CPU > 90%." But what if 70% CPU on a Tuesday morning is unusual for your app? Fixed rules miss subtle, context-dependent signals.

The ML Solution

Anomaly detection models learn your system's unique rhythm — day vs. night, weekday vs. weekend, before vs. after a deployment. They raise an alert only when behavior is **statistically unusual**, not just above a number.

Real-world example: Netflix uses anomaly detection to catch streaming degradation in specific regions within seconds — long before users flood support with complaints.

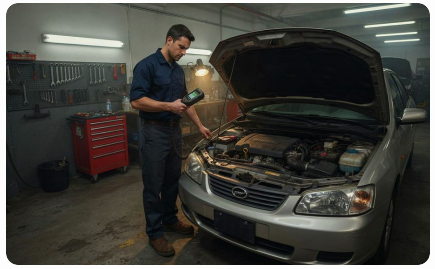
Predictive Analytics — From Reactive to Proactive

Anomaly detection tells you something is wrong *right now*. Predictive analytics tells you something *will go wrong* — giving your team time to prevent the incident entirely.



Capacity Forecasting

Predict when your servers will run out of capacity based on historical growth trends — and scale before users feel the pain.



Failure Prediction

Predict hardware failures or service degradation days in advance, like a check-engine light for your infrastructure.



Incident Forecasting

Identify patterns that historically precede major incidents and trigger automated remediation workflows proactively.

PRACTICAL ALGORITHMS

The Three Workhorses of ML in AIOps

You don't need a PhD in statistics to apply ML in operations. Three categories of algorithms handle the vast majority of AIOps use cases. Think of them as different tools in a toolbox — each best suited for a specific type of problem.



Classification — Is This Normal or Not?

Classification algorithms answer **yes/no questions**: Is this log entry an error? Is this deployment likely to fail? Is this alert real or noise? Think of it like an airport security scanner — it categorizes every bag as "safe" or "needs inspection" based on learned patterns.

Example Algorithm

Random Forest, Logistic Regression, SVM

AIOps Use Case

Classifying alerts as "critical," "warning," or "noise" to reduce alert fatigue

Real-World Analogy

Spam filter in your email — learned from millions of examples what "spam" looks like



Regression — Predicting Numbers Over Time

Regression answers **"how much?" or "when?"** questions. Instead of a yes/no, you get a number — like predicting that your database will hit 95% disk capacity in 4 days, giving you a precise window to act.

Common algorithms: Linear Regression, Gradient Boosting, ARIMA for time-series data.

Real-world analogy: A weather forecast doesn't just say "it will rain" — it says "35mm of rain expected by Thursday." Regression gives your ops team that same precision.

Disk Usage Forecast

Predict exact date storage will be exhausted

Response Time Trend

Forecast API latency under increasing load

Cost Prediction

Estimate next month's cloud spend from usage patterns

Clustering — Finding Patterns You Didn't Know Existed

Clustering groups similar things together **without being told what to look for**. It's the algorithm that looks at 10,000 alerts and says, "these 3,000 alerts are probably all caused by the same underlying issue." This is called **alert correlation** — one of the most powerful AIOps capabilities.



Alert Grouping

Cluster 500 noisy alerts into 5 meaningful incident groups, so engineers see root causes — not a flood of notifications.



Log Pattern Mining

Automatically discover recurring log templates in unstructured log data without writing a single regex pattern.



Service Dependency Maps

Cluster services by communication patterns to automatically build topology maps for impact analysis.



GITHUB FOR ML

Deploying ML Models with Version Control on GitHub

A machine learning model is just code — and like code, it needs to be **versioned, reviewed, tested, and deployed** in a controlled way. GitHub gives ML teams all the tools they already use for software, applied to the full ML lifecycle: from data pipelines to trained model artifacts.

Why Version Control for ML Models?

Imagine your anomaly detection model starts flagging too many false positives after a system update. Without version control, you'd struggle to answer: *Which model version is running in production? What changed? Can we roll back?* With GitHub, these questions have instant answers.

1

Code & Config

Training scripts, hyperparameters, and feature engineering — all tracked in Git

2

Model Artifacts

Trained model files stored via Git LFS or linked to GitHub Releases

3

Deploy & Rollback

Tag releases, promote to production, and roll back to a previous model in minutes

Key principle: treat your model like a product release — every change should go through a pull request with peer review and automated testing.

Structuring an ML Repository on GitHub

A well-organized GitHub repo for an ML project makes collaboration easy and deployment reliable. Here's a structure that real teams use in production AIOps projects:

📁 Recommended Structure

```
/data          → raw & processed datasets
/notebooks     → exploratory analysis
/src           → training & inference code
/models        → saved model artifacts
/tests         → unit & integration tests
/.github       → Actions workflows
README.md      → setup & usage guide
```

Why It Matters

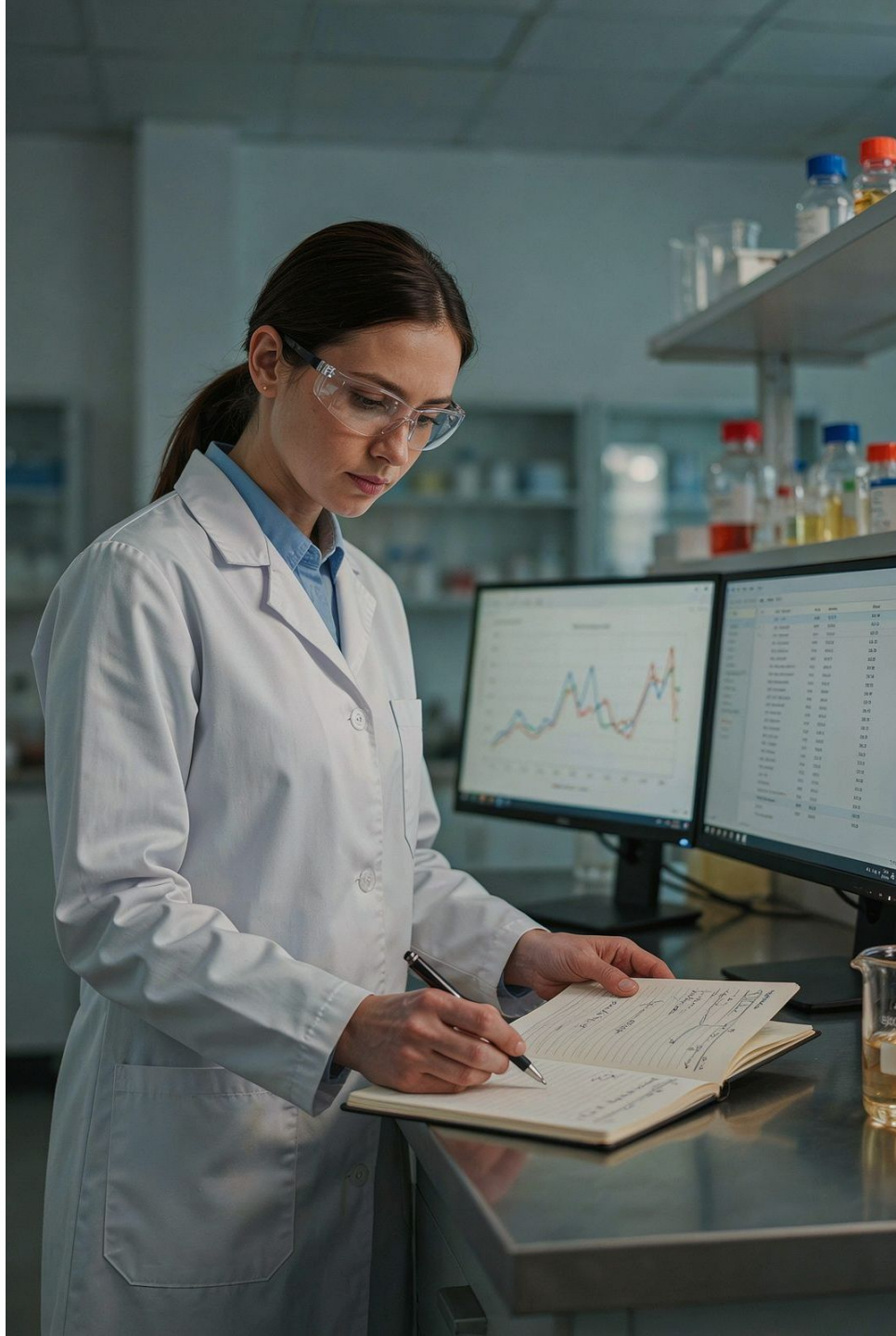
Separating concerns means a data scientist can update a notebook without touching the deployment pipeline. A DevOps engineer can update the CI/CD workflow without touching the model code. **Less conflict, faster iteration.**

GitHub's branch protection rules ensure that no model goes to production without passing automated tests and at least one code review.

MODEL TRACKING

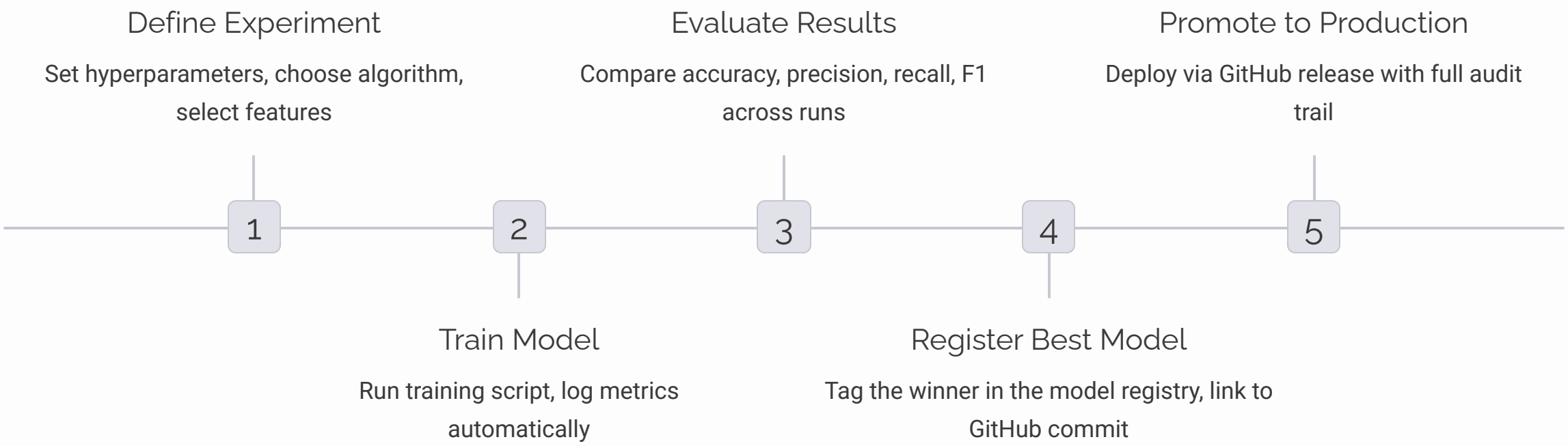
Model Tracking, Experimentation & Evaluation

Training a model once is easy. Knowing **which version of your model performs best — and why** — is the hard part. Model tracking is the discipline of recording every experiment so you can reproduce results, compare approaches, and confidently promote the right model to production.



The Experiment Tracking Workflow

Think of experiment tracking like a **scientific lab notebook**. Every time you train a model — whether you're testing a new algorithm, different features, or tuned hyperparameters — you log the inputs and outputs so you can compare runs side by side. Tools like **MLflow** or **DVC** integrate seamlessly with GitHub.



Evaluation Metrics That Actually Matter in AIOps

Accuracy alone is a misleading metric for AIOps models. In a system where 99% of events are normal, a model that always says "normal" has 99% accuracy — but is completely useless. The metrics that matter are those that capture **how well the model handles the rare, critical events.**



Precision

Of all alerts raised, how many were real incidents? High precision = fewer false alarms waking up engineers at 3am.



Recall

Of all real incidents, how many did we catch? High recall = we don't miss actual outages.



F1 Score

The harmonic mean of precision and recall — the single number that balances both concerns for AIOps use cases.



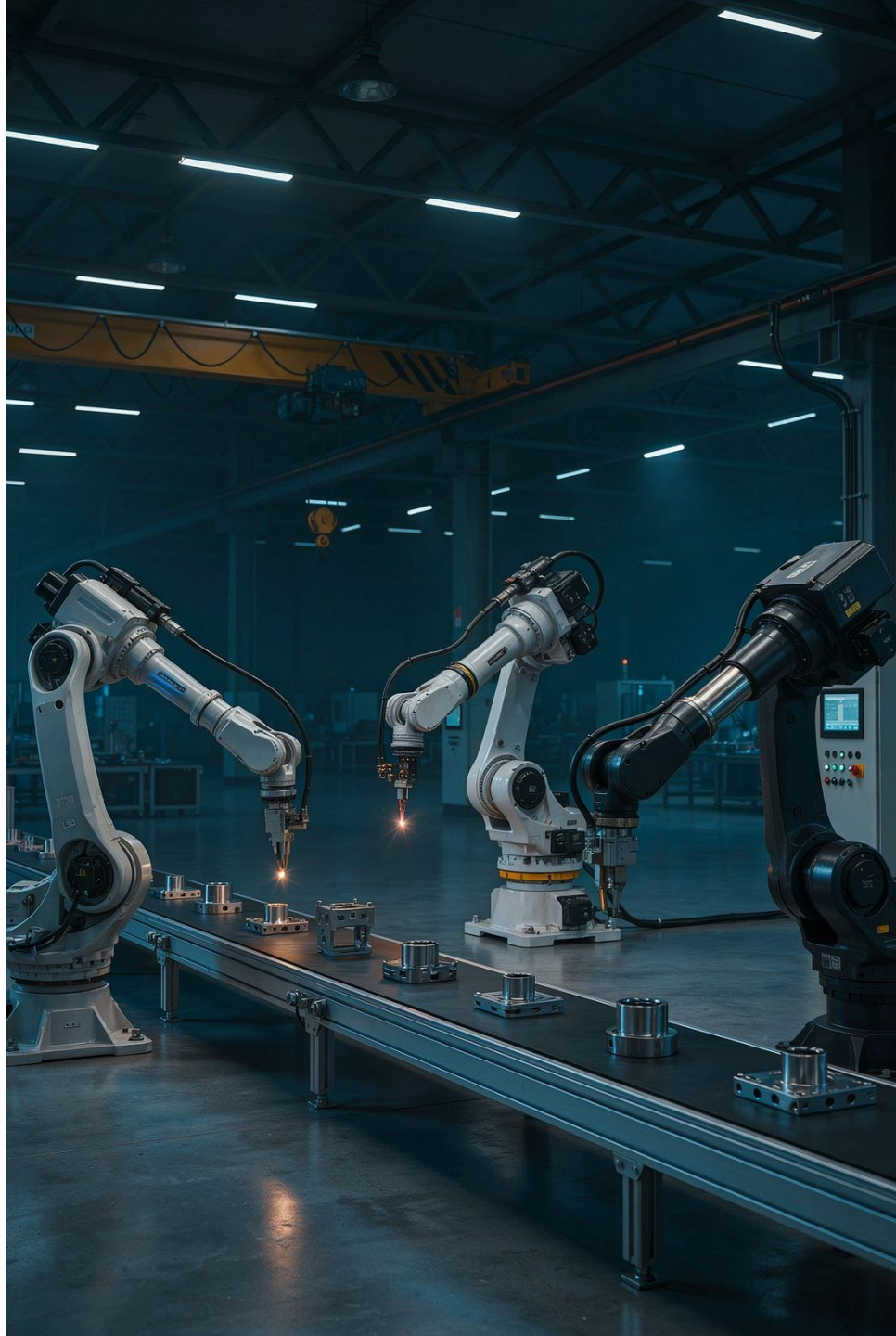
Inference Latency

How fast can the model score a new event? In real-time AIOps, a slow model is as bad as a wrong one.

GITHUB ACTIONS

GitHub Actions — Your ML Automation Engine

GitHub Actions lets you **automate the entire ML lifecycle** — from retraining a model when new data arrives, to running evaluation tests before any model goes live, to deploying a new version the moment it passes all checks. It's the "factory floor" that keeps your ML pipeline running without manual intervention.



Automating ML Workflows with GitHub Actions

Just as CI/CD automates software delivery, GitHub Actions automates the **continuous training and delivery of ML models**. Here's a typical workflow for an AIOps anomaly detection model:

⚙️ Sample Workflow Triggers

- **On schedule:** Retrain the model every Sunday night on fresh data
- **On push:** Run unit tests when training code changes
- **On pull request:** Evaluate model performance vs. current production baseline
- **On data update:** Trigger pipeline when new labeled incident data lands in the repo

What Each Step Does

- **Data Validation**

  Check that incoming data meets expected schema and quality thresholds

- **Model Training**

  Execute training script, log metrics to MLflow or GitHub Summary

- **Evaluation Gate**

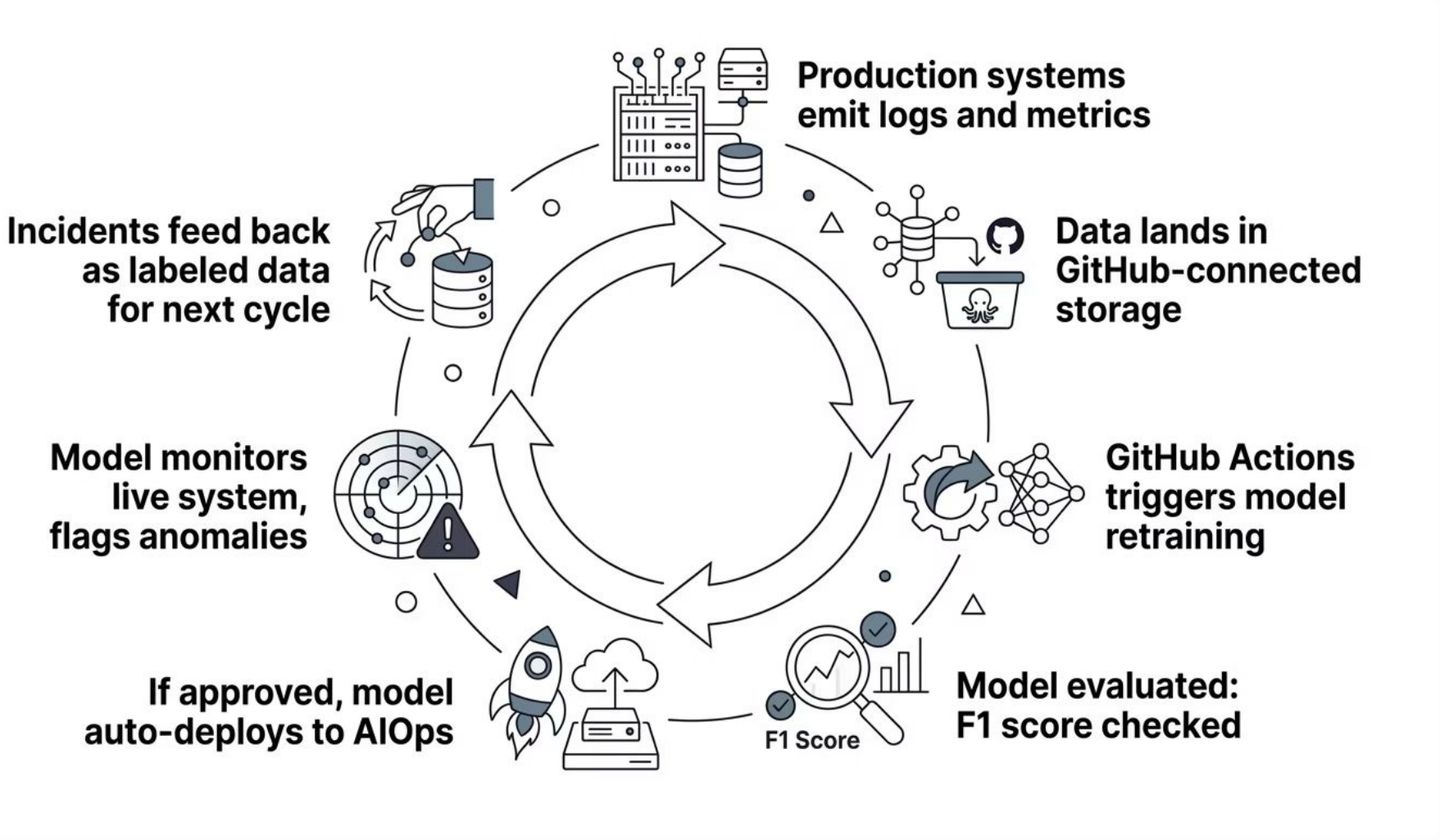
  Block deployment if F1 score drops below defined threshold

- **Auto Deploy**

  Push approved model to production endpoint automatically

Putting It All Together — The Full AIOps ML Loop

Here's how all the pieces connect in a real-world AIOps scenario at a company like an e-commerce platform running on GitHub. The entire loop — from data to insight to action — runs automatically, with engineers reviewing and approving at key checkpoints.



Module 3 — Key Takeaways

You now have a complete picture of how machine learning powers modern AIOps — and how GitHub ties it all together into a reliable, repeatable workflow.

🧠 ML is the brain of AIOps

Anomaly detection and predictive analytics replace brittle threshold rules with adaptive, learning systems.

🔧 Three algorithms cover most cases

Classification, regression, and clustering solve the majority of real-world AIOps problems.

📦 Treat models like software

Version control, code review, and structured repos on GitHub bring discipline to ML development.

⚡ Automate everything with Actions

GitHub Actions closes the loop — models retrain, evaluate, and deploy without manual toil, just like a production software pipeline.

